

Name: Sk Arshad Basha

RegNo: 192312336

1. Design a pseudocode algorithm to read a dataset from a file and preprocess it using array and Data Frame operations. The algorithm should handle missing values, remove duplicate records, normalize numerical columns, and standardize categorical values. Include steps for indexing, filtering invalid entries, and saving the cleaned dataset. Ensure the algorithm clearly defines input, processing steps, and output while maintaining computational efficiency.

Objective

To read a dataset from a file and preprocess it using array and DataFrame operations. The algorithm handles missing values, duplicate records, normalization of numerical columns, standardization of categorical values, indexing, filtering, and saving the cleaned dataset.

Input

- Dataset file input.csv
- Numerical columns
- Categorical columns
- Valid value constraints

Output

- Cleaned and normalized dataset cleaned_data.csv

Pseudocode

ALGORITHM PREPROCESS_DATASET

INPUT:

File_Name

Numerical_Columns

Categorical_Columns

Valid_Ranges

OUTPUT:

Cleaned_Dataset

BEGIN

1. LOAD DATA

TRY

 DataFrame ← READ File_Name

CATCH File_Error

 DISPLAY "Unable to read file"

 STOP

END TRY

2. VALIDATE DATA

IF DataFrame is empty THEN

 DISPLAY "Dataset is empty"

 STOP

END IF

3. CLEAN COLUMN NAMES

REMOVE leading and trailing spaces

STANDARDIZE column names

END FOR

END

Efficiency

- Vectorized array operations are preferred over loops.
- Only required columns are processed.
- Duplicate checking is performed once.
- Missing-value replacement uses aggregation functions such as median and mode.
- Indexing and filtering are performed using DataFrame operations.

Conclusion

This algorithm produces a clean dataset by systematically handling missing values, duplicates, invalid entries, numerical normalization, and categorical standardization while maintaining computational efficiency.

2. Write a detailed pseudocode algorithm to compute statistical measures such as mean, median, standard deviation, and variance from a large dataset. The algorithm should demonstrate how to efficiently use array-based operations, apply filtering to exclude unwanted values, and perform aggregation. Include steps for validating the dataset and handling edge cases such as empty arrays or extreme outliers.

Objective

To calculate mean, median, standard deviation, and variance from a large dataset using efficient array operations and aggregation.

Input

- Large numerical dataset
- Filtering conditions
- Valid numerical range

Output

- Mean
- Median
- Variance
- Standard deviation

Pseudocode

ALGORITHM STATISTICAL_ANALYSIS

INPUT:

Dataset

Minimum_Value

Maximum_Value

OUTPUT:

Mean

Median

Variance

Standard_Deviation

BEGIN

1. LOAD DATA

Data $\leftarrow$ READ numerical dataset

2. VALIDATE DATA

IF Data is empty THEN

 DISPLAY "Dataset is empty"

 STOP

END IF

3. CONVERT TO NUMERICAL ARRAY

Array $\leftarrow$ CONVERT Data to numerical array

END

Efficiency

For a large dataset:

- Use array-based/vectorized operations.
- Avoid unnecessary Python loops.
- Process data in chunks if the dataset cannot fit into memory.

Conclusion

The algorithm efficiently calculates statistical measures by first validating and filtering the dataset and then applying array-based aggregation. It also handles empty datasets, missing values, invalid values, and extreme observations.

3. Develop a pseudocode algorithm to combine two or more DataFrames containing related data. The algorithm must include steps for aligning indices, merging datasets based on a common key, removing duplicates, and validating the integrity of the merged data. Incorporate grouping and sorting operations,

and ensure that constraints such as consistency and completeness of records are maintained.

Objective

To combine two or more related DataFrames while maintaining index consistency, common-key integrity, completeness, and correctness.

Input

- DataFrame 1
- DataFrame 2
- Common key

Output

- Validated merged DataFrame

Pseudocode

ALGORITHM COMBINE_DATAFRAMES

INPUT:

 DataFrame1

 DataFrame2

 Common_Key

OUTPUT:

 Merged_DataFrame

BEGIN

1. LOAD DATAFRAMES

 READ DataFrame1

 READ DataFrame2

2. VALIDATE DATAFRAMES

 IF either DataFrame is empty THEN

 DISPLAY "Invalid or empty DataFrame"

 STOP

END IF

3. CHECK COMMON KEY

IF Common_Key does not exist in both DataFrames THEN

 DISPLAY "Common key not found"

 STOP

END IF

END

Integrity Checks

CHECK duplicate Customer_ID

CHECK missing Customer_ID

CHECK unmatched records

CHECK number of records

CHECK data types

CHECK final index

Conclusion

The algorithm combines related Data Frames using a common key while preserving valid records. Index alignment, duplicate removal, grouping, aggregation, sorting, and final validation ensure that the merged dataset remains consistent and complete.

4. Create a pseudocode algorithm that filters a dataset based on multiple conditions (e.g., value ranges, categories) and then sorts the filtered results. The algorithm should utilize indexing techniques, logical conditions, and efficient sorting methods. Include steps for handling large datasets and optimizing performance while ensuring that the output meets all filtering constraints.

Objective

To filter a dataset using multiple conditions and efficiently sort the resulting records.

Input

- Dataset
- Numerical range
- Required categories
- Sorting column

Output

- Filtered and sorted dataset

Pseudocode

ALGORITHM FILTER_AND_SORT

INPUT:

Dataset

Minimum_Value

Maximum_Value

Required_Categories

Sort_Column

OUTPUT:

Filtered_Sorted_Data

BEGIN

1. VALIDATE DATASET

IF Dataset is empty THEN

 DISPLAY "Dataset is empty"

 STOP

END IF

2. SELECT REQUIRED COLUMNS

 Keep only columns required for analysis

END

Handling Large Datasets

For large datasets:

- Select only required columns.
- Filter before sorting.
- Use vectorized Boolean conditions.
- Process the dataset in chunks when necessary.
- Avoid unnecessary copies.
- Sort only the filtered records.

The important principle is:

Large Dataset



Filter First



Smaller Dataset



Sort

This is more efficient than sorting the complete dataset before filtering.

Conclusion

The filtering and sorting algorithm uses indexing, Boolean conditions, logical operators, and efficient sorting. Applying filters before sorting reduces the number of records that must be sorted and therefore improves performance for large datasets.

5. Write a pseudocode algorithm for a complete data analysis workflow that includes reading data from files, converting it into DataFrames, performing grouping and aggregation, and generating plots. The algorithm should specify steps for handling file input/output errors, applying transformations, and producing summarized results suitable for visualization. Ensure clarity in each stage from data loading to final output generation.

Objective

To design a complete workflow that reads data from files, converts it into DataFrames, cleans and transforms the data, performs grouping and aggregation, and generates plots.

Input

- One or more data files
- User-defined analysis conditions

Output

- Clean DataFrame
- Aggregated results
- Visualization plots
- Summary output

Pseudocode

ALGORITHM COMPLETE_DATA_ANALYSIS

INPUT:

File_List

Grouping_Column

Aggregation_Column

Filter_Condition

OUTPUT:

Clean_Data

Summary_Data

Plot

BEGIN

1. INITIALIZE

Create empty list DataFrames

2. READ FILES

FOR each File in File_List

```

TRY
    Data ← READ File
    IF Data is empty THEN
        DISPLAY "File is empty"
        CONTINUE
    END IF
    ADD Data to DataFrame_List
CATCH File_Input_Error
    DISPLAY "File cannot be read"
CATCH Format_Error
    DISPLAY "Invalid file format"
END TRY
END FOR

```

3. CHECK DATA

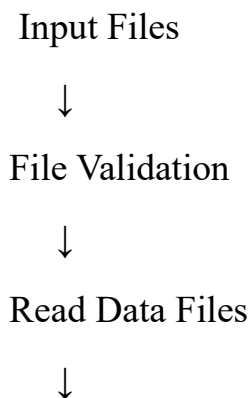
```

IF DataFrame_List is empty THEN
    DISPLAY "No valid data available"
    STOP
END IF

```

END

Complete Workflow



Create DataFrames



Combine Data



Clean Missing Data



Remove Duplicates



Transform Data



Apply Filtering



Group Records



Aggregate



Sort Data



Validate Results



Generate Plot



Save Outputs

Error Handling

The algorithm checks for:

- File not found.

- Empty files.
- Invalid file formats.
- Corrupted data.
- Missing columns.
- Incorrect data types.
- Missing values.
- Duplicate records.

Aggregation Example

If sales data is grouped by category:

GROUP BY Category

FOR each Category

Total_Sales $\leftarrow$ SUM(Sales)

Average_Sales $\leftarrow$ MEAN(Sales)

Number_of_Records $\leftarrow$ COUNT(Sales)

END FOR

The resulting summary can be directly used for visualization.

Conclusion

The complete data-analysis workflow provides a systematic process from file loading to final visualization. It handles file errors, converts data into DataFrames, cleans and transforms records, applies filtering, performs grouping and aggregation, sorts the results, validates the output, and generates meaningful plots. This approach provides an efficient and reliable method for analyzing large datasets.